

Kế Hoạch Triển Khai Backend — Hệ Thống Đấu Giá Trực Tuyến

Phạm vi phân tích: 31 file Java trong các package dao, dao.impl, model, factory, db, service, util; đối chiếu thêm với pom.xml, database.properties, schema.sql và cấu trúc thư mục hiện có.

Ghi chú: link Google Drive được nêu trong đề bài không truy cập trực tiếp được từ môi trường làm việc hiện tại, vì vậy nhận định bám trên mã nguồn và schema đang có. Source Java hiện tại biên dịch được bằng javac, nhưng vẫn tồn tại các rủi ro runtime/thiết kế đáng kể.

Mục Lục

1. Đánh Giá Cấu Trúc Hiện Tại	3
Nhận xét theo cấu trúc package	3
Các phát hiện kiến trúc quan trọng	3
File/class đang ôm quá nhiều trách nhiệm	4
Các package/class nên bổ sung	4
Đánh giá tổng thể: điểm mạnh và điểm yếu	4
2. Kế Hoạch Chi Tiết Từng File	6
Package dao	6
AuctionDAO.java	6
BidTransactionDAO.java	6
ItemDAO.java	7
UserDAO.java	7
Package dao.impl	8
AuctionDAOImpl.java	8
BidTransactionDAOImpl.java	9
ItemDAOImpl.java	9
UserDAOImpl.java	10
Package model	10
Entity.java	10
User.java	11
Admin.java	12
Bidder.java	12

Seller.java	13
Item.java	13
Art.java	14
Collectible.java	14
Electronics.java	15
Fashion.java	16
Vehicle.java	16
Auction.java	17
BidTransaction.java	18
AutoBidding.java	18
Package factory	19
ItemFactory.java	19
ArtFactory.java	19
CollectibleFactory.java	20
ElectronicsFactory.java	21
FashionFactory.java	21
VehicleFactory.java	22
Package db	22
DatabaseManager.java	22
Package service	23
AuctionManager.java	23
Package util	24
DatabaseSetup.java	24

3. Lộ Trình Học & Làm Theo Thứ Tự 25

Thứ tự nên code các file	25
Lộ trình học kiến thức theo từng giai đoạn	25
Kết luận ngắn	26

1. Đánh Giá Cấu Trúc Hiện Tại

Nhìn tổng thể, codebase hiện tại có bộ khung package khá đúng hướng cho một backend Java thuần: có phân tách dao, model, factory, db, service, util và resource database. Tuy nhiên phần hiện thực đang thiên về minh họa OOP/design pattern hơn là một backend đồng nhất cho production hoặc demo cuối kỳ.

Điểm tích cực là nhóm đã nghĩ tới DAO Pattern, Factory Method, Observer, transaction và scheduler. Điểm cần xử lý gấp là sự lệch pha giữa model, DAO và schema.sql; nếu không chốt lại mô hình dữ liệu, các tầng phía trên sẽ tiếp tục phát sinh lỗi dây chuyền.

Package	Số file	Nhận xét nhanh
dao	4	Có contract rõ ràng, nhưng thiếu AutoBiddingDAO và contract phụ trợ.
dao.impl	4	Tách implementation khỏi interface là hợp lý; cần mapper/helper để tránh class quá dài.
model	14	Domain khá phong phú nhưng lẫn nhiều hành vi UI console và string literal.
factory	6	Có pattern nhưng hơi nặng tay cho số lượng item hiện tại.
db	1	Thiếu config, pooling và tách quản lý transaction bài bản.
service	1	Một class duy nhất đang ôm hầu hết luồng nghiệp vụ.
util	1	Mới có bootstrap schema, chưa có util hỗ trợ bảo mật/validation/JDBC.
observer, client, server	3	observer mới có interface; client/server là placeholder trống, chưa tạo giá trị thực.

Nhận xét theo cấu trúc package

- Khung package cơ bản là hợp lý cho đồ án Java thuần: model chứa domain, dao chứa persistence, service chứa orchestration, util/db chứa hạ tầng.
- Subpackage dao.impl là hợp lý nếu mục tiêu là tách interface khỏi JDBC implementation; nếu sau này chỉ dùng MySQL, có thể đổi tên thành dao.mysql để rõ ngữ cảnh hơn.
- Factory package đang tồn tại đúng tinh thần design pattern, nhưng mức chi tiết hiện tại khiến code verbose hơn nhu cầu thực tế.
- Thiếu các package rất quan trọng cho backend nghiêm túc: enum, exception, validation, mapper, config và test.
- Các package client/server hiện chỉ có class upload rỗng, nên hoặc bỏ đi, hoặc tách thành module riêng khi thật sự triển khai network layer.

Các phát hiện kiến trúc quan trọng

Mức độ	Phát hiện	Tác động
Rất cao	Item/ItemDAOImpl dùng current_price trong bảng items, nhưng schema.sql đặt current_price ở bảng auctions.	Dễ lỗi SQL runtime và trùng trách nhiệm dữ liệu giữa Item và Auction.
Rất cao	Code dùng ACTIVE/CLOSED/CANCELLED, nhưng schema dùng OPEN/RUNNING/FINISHED/PAID/CANCELED.	Insert, query và scheduler auction có thể sai ngay khi chạy với database thật.
Cao	AuctionManager ôm tạo phiên, đặt giá, monitor, observer, settlement và raw SQL transaction.	Khó test, khó mở rộng và rủi ro cao khi sửa logic đấu giá.

Cao	Password và thông tin DB đang để plain text/hardcode ở source và resource.	Rủi ro bảo mật, khó cấu hình môi trường khác nhau.
Trung bình	Model chứa nhiều System.out.println và phương thức mang tính giao diện console.	Lẫn domain layer với presentation, gây khó tái sử dụng backend.
Trung bình	Có AutoBidding model và bảng auto_biddings nhưng thiếu DAO/service đi kèm.	Chức năng auto-bid đang dở dang, khó hoàn thiện đúng hạn nếu không bổ sung lớp mới.
Trung bình	target/out chứa artifact không còn source tương ứng như Main, BaseDAO, JDBCConnectionTest.	Repo hygiene kém, dễ làm nhóm hiểu nhầm code nào là source thật.

File/class đang ôm quá nhiều trách nhiệm

- AuctionManager.java là class quá tải trách nhiệm rõ nhất: vừa quản lý cache active auctions, vừa điều phối bidding, vừa chạy scheduler, vừa notify observer, vừa tự viết SQL settlement. Đây là ứng viên số một cần tách nhỏ.
- ItemDAOImpl.java và UserDAOImpl.java cũng đang mang quá nhiều logic mapping thủ công, switch/instanceof và bind cột. Trong ngắn hạn có thể chấp nhận, nhưng về lâu dài nên tách mapper/helper riêng.
- DatabaseManager.java hiện vừa load driver, vừa giữ cấu hình hardcode, vừa cấp connection, vừa quản lý transaction. Nếu dùng lâu dài, nên tách DatabaseConfig và TransactionHelper.
- Các model như User/Item và các lớp con đang gánh thêm trách nhiệm hiển thị console, vốn không thuộc backend domain model.

Các package/class nên bổ sung

Đề xuất	Lý do
com.team4.enums.AuctionStatus, UserRole, ItemCategory	Loại bỏ String literal và đồng bộ tuyệt đối giữa model, DAO, service, database.
com.team4.dao.AutoBiddingDAO / com.team4.dao.impl.AutoBiddingDAOImpl	Hoàn thiện phần auto-bid vốn đã có model và bảng dữ liệu.
com.team4.service.UserService, ItemService, BiddingService, SettlementService, AutoBiddingService	Tách AuctionManager thành các service đúng SRP, dễ test và dễ sửa.
com.team4.util.PasswordHasher, ValidationUtil, JdbcUtil, DateTimeUtil	Bổ sung các tiện ích backend đang thiếu: hash mật khẩu, validate, đóng gói JDBC, xử lý thời gian.
com.team4.exception.BusinessException, ValidationException, ConcurrencyException, NotFoundException	Giúp service/DAO báo lỗi rõ ràng thay vì chỉ return false hoặc null.
com.team4.mapper.UserMapper, ItemMapper, AuctionMapper	Giảm độ phức tạp của DAO implementation khi mapping ResultSet sang object.
src/test/java + test classes	Dự án hiện không có test; đây là khoảng trống lớn trước khi demo hoặc nộp đồ án.

Đánh giá tổng thể: điểm mạnh và điểm yếu

Điểm mạnh

- Đã có ý thức phân tầng package tương đối đúng cho backend Java thuần.
- Nhóm đã chủ động áp dụng nhiều khái niệm kiến trúc quan trọng: DAO, Factory, Observer, transaction.

- Schema.sql và resource cấu hình đã xuất hiện sớm, thuận lợi cho việc dựng môi trường local.
- Model nghiệp vụ phong phú, bám được domain đấu giá ở mức khung sườn.

Điểm yếu

- Chưa có sự thống nhất dữ liệu giữa model, DAO và database schema.
- Service layer quá tập trung vào một class duy nhất.
- Bảo mật còn yếu: hardcode credential, plain text password, thiếu exception/validation rõ ràng.
- Chưa có test, chưa có migration bài bản, repo còn chứa artifact build gây nhiễu.
- Model lẫn nhiều code demo console, khiến backend khó tái sử dụng cho UI khác.

2. Kế Hoạch Chi Tiết Từng File

Phần này coi mỗi file như đang ở trạng thái trắng và lập kế hoạch implement từ đầu. Riêng subpackage dao.impl được đưa vào vì đây là phần hiện thực trực tiếp của package dao.

Package dao

Package interface cho tầng truy cập dữ liệu. Tốt cho việc tách contract, nhưng hiện vẫn thiếu AutoBiddingDAO và các enum/exception hỗ trợ.

AuctionDAO.java

1. Ý nghĩa & vai trò:

Định nghĩa contract truy cập dữ liệu cho thực thể Auction. Nó là ranh giới giữa tầng service và tầng SQL, đặc biệt quan trọng cho chức năng đặt giá đồng thời.

2. Kiến thức cần học trước khi code:

- DAO Pattern, interface trong Java và nguyên tắc tách contract khỏi implementation.
- JDBC cơ bản: Connection, PreparedStatement, ResultSet, Timestamp.
- Optimistic locking hoặc điều kiện cập nhật có kiểm tra trạng thái để xử lý concurrent bidding.
- Thiết kế query theo index và enum trạng thái phiên đấu giá.

3. Nội dung cần implement:

- findById(String id): Tìm phiên đấu giá theo ID.
- findByItemId(String itemId): Tìm phiên đấu giá gắn với một item cụ thể.
- findAll(): Lấy danh sách toàn bộ phiên đấu giá.
- findByStatus(String status): Lọc phiên theo trạng thái để phục vụ màn hình hoặc scheduler.
- insert(Auction auction): Tạo mới một phiên đấu giá.
- update(Auction auction): Cập nhật thông tin phiên như giá hiện tại hoặc người dẫn đầu.
- updateStatus(String auctionId, String newStatus): Chuyển trạng thái phiên khi đóng, thanh toán hoặc hủy.
- placeBid(String auctionId, String bidderId, BigDecimal newBidPrice): Cập nhật giá đấu mới theo cơ chế chống race condition.

4. Những điều cần lưu ý khi code:

- Tên trạng thái trong interface phải thống nhất tuyệt đối với enum trong model và giá trị enum của MySQL.
- Không nên để DAO quyết định rule nghiệp vụ như ai được bid; DAO chỉ kiểm tra điều kiện dữ liệu cần thiết.
- Method đặt giá nên dùng BigDecimal và có cơ chế trả về rõ ràng khi cập nhật thất bại do tranh chấp.

5. Thứ tự ưu tiên:

Bước 4 — định nghĩa sau khi model Auction và status enum đã chốt, trước khi viết AuctionDAOImpl.

BidTransactionDAO.java

1. Ý nghĩa & vai trò:

Định nghĩa contract lưu lịch sử đặt giá. Đây là nguồn audit để truy vết ai đã đặt giá nào, lúc nào và phục vụ báo cáo/lịch sử giao dịch.

2. Kiến thức cần học trước khi code:

- DAO Pattern và audit logging.
- JDBC, ORDER BY, LIMIT và cách tối ưu truy vấn lịch sử bằng index.
- Khái niệm immutable record cho dữ liệu giao dịch.

- Phân biệt dữ liệu hiện tại của auction với dữ liệu lịch sử của bids.

3. Nội dung cần implement:

- insert(BidTransaction transaction): Lưu một lần đặt giá mới.
- findByAuctionId(String auctionId): Lấy toàn bộ lịch sử của một phiên để hiển thị hoặc phân tích.
- findByBidderId(String bidderId): Lấy lịch sử các lần bid của một người dùng.
- getHighestBid(String auctionId): Lấy giao dịch có giá cao nhất của phiên.

4. Những điều cần lưu ý khi code:

- Không dùng bảng lịch sử làm nguồn duy nhất để xác định trạng thái hiện tại nếu bảng auctions đã lưu current price.
- Nên hỗ trợ pagination khi dữ liệu lớn để tránh kéo toàn bộ lịch sử về bộ nhớ.
- Timestamp nên do server/database sinh ra để tránh lệch múi giờ từ client.

5. Thứ tự ưu tiên:

Bước 4 — định nghĩa sau AuctionDAO vì nó phụ thuộc vào khái niệm phiên đấu giá nhưng trước DAO implementation.

ItemDAO.java

1. Ý nghĩa & vai trò:

Định nghĩa contract CRUD cho vật phẩm đấu giá. File này chịu trách nhiệm mô tả cách lấy item theo owner, category và ID mà không dính tới chi tiết SQL.

2. Kiến thức cần học trước khi code:

- DAO Pattern và CRUD cơ bản cho thực thể cha có nhiều lớp con.
- Thiết kế truy vấn lọc theo category, owner, keyword.
- Chiến lược ánh xạ kế thừa xuống MySQL.
- Nguyên tắc giữ item là metadata sản phẩm, không nhầm với dữ liệu phiên đấu giá.

3. Nội dung cần implement:

- findById(String id): Lấy chi tiết item theo ID.
- findAll(): Lấy toàn bộ item.
- findByCategory(String category): Lọc theo danh mục.
- findByOwnerId(String ownerId): Lọc theo người sở hữu hoặc người đăng bán.
- insert(Item item): Tạo mới item.
- update(Item item): Cập nhật metadata của item.
- delete(String id): Xóa item khi chưa hoặc không còn được sử dụng.

4. Những điều cần lưu ý khi code:

- Cần thống nhất item có giữ currentPrice hay không; nếu currentPrice thuộc Auction thì ItemDAO không được đụng vào cột đó.
- Nếu dùng single-table inheritance, phải kiểm soát chặt nullability của các cột đặc thù.
- DAO không nên chứa switch business logic lớn cho từng loại item; nên có mapper hoặc factory hỗ trợ.

5. Thứ tự ưu tiên:

Bước 4 — làm sau khi Item và các lớp con đã được thiết kế xong.

UserDAO.java

1. Ý nghĩa & vai trò:

Định nghĩa contract truy cập dữ liệu người dùng. Đây là đầu mối cho đăng nhập, tra cứu hồ sơ và cập nhật số dư hoặc thông tin profile.

2. Kiến thức cần học trước khi code:

- DAO Pattern và truy vấn theo khóa duy nhất như username/email.
- Password hashing, salt và nguyên tắc không lưu plain text password.
- Role-based design với Admin, Seller, Bidder.
- Unique constraint, transaction và validation dữ liệu người dùng.

3. Nội dung cần implement:

- findById(String id): Tìm người dùng theo ID.
- findByUsername(String username): Tìm người dùng theo username khi đăng nhập.
- insert(User user): Tạo tài khoản mới.
- update(User user): Cập nhật thông tin hồ sơ hoặc số dư.
- findAll(): Lấy danh sách toàn bộ người dùng.

4. Những điều cần lưu ý khi code:

- Không expose password đã hash qua các luồng không cần thiết.
- Role nên dùng enum thay vì String để tránh sai chính tả giữa code và database.
- Nên tách cập nhật profile và cập nhật số dư thành các use case khác nhau để tránh ghi đè ngoài ý muốn.

5. Thứ tự ưu tiên:

Bước 4 — định nghĩa sau khi User và các lớp con ổn định, trước UserDaoImpl và AuthenticationService.

Package dao.impl

Subpackage triển khai JDBC thực tế. Cấu trúc này hợp lý cho Java thuần, nhưng cần mapper/helper để tránh class phình to.

AuctionDAOImpl.java

1. Ý nghĩa & vai trò:

Triển khai SQL thật cho AuctionDAO. Đây là nơi chuyển dữ liệu giữa bảng auctions và object Auction, đồng thời thực thi cơ chế đặt giá an toàn khi có nhiều người bid.

2. Kiến thức cần học trước khi code:

- PreparedStatement, ResultSet mapping và chuyển đổi Timestamp <-> LocalDateTime.
- Optimistic locking hoặc điều kiện WHERE để chống lost update.
- SQLException handling, transaction boundary và logging thay cho printStackTrace.
- Cách tổ chức SQL constants và helper mapper để class không bị dài quá mức.

3. Nội dung cần implement:

- mapRowToAuction(ResultSet rs): Chuyển một row từ MySQL thành Auction.
- findById(...), findByItemId(...), findAll(), findByStatus(...): Các truy vấn đọc dữ liệu phiên đấu giá.
- insert(Auction auction): Lưu phiên mới vào bảng auctions.
- update(Auction auction): Cập nhật giá hiện tại, bidder dẫn đầu hoặc thời gian kết thúc.
- updateStatus(...): Chuyển trạng thái phiên.
- placeBid(...): Thực thi câu lệnh update có điều kiện để chỉ một bid hợp lệ được ghi nhận.

4. Những điều cần lưu ý khi code:

- Trạng thái đang là điểm lịch nghiêm trọng giữa code và schema; phải chốt enum trước khi viết SQL.
- Không nên swallow lỗi rồi trả false chung chung; nên có exception hoặc result type rõ lý do thất bại.
- Query đặt giá phải kiểm tra cả trạng thái, thời gian và mức giá mới để tránh bid vào phiên hết hạn.

5. Thứ tự ưu tiên:

Bước 5 — viết sau AuctionDAO, Auction model và schema đã thống nhất.

BidTransactionDAOImpl.java

1. Ý nghĩa & vai trò:

Triển khai SQL cho lịch sử đặt giá. File này chịu trách nhiệm ghi audit trail và đọc lại lịch sử phục vụ báo cáo hoặc trang chi tiết phiên đấu giá.

2. Kiến thức cần học trước khi code:

- ResultSet mapping, Timestamp handling và thứ tự sắp xếp dữ liệu theo thời gian.
- Index tối ưu cho truy vấn theo auction_id và bidder_id.
- Nguyên tắc immutable đối với dữ liệu giao dịch.
- Kỹ thuật đọc dữ liệu theo batch hoặc pagination khi lịch sử lớn.

3. Nội dung cần implement:

- mapRowToTransaction(ResultSet rs): Map row thành BidTransaction.
- insert(BidTransaction transaction): Lưu một lần bid vào bảng lịch sử.
- findByAuctionId(String auctionId): Đọc lịch sử theo phiên.
- findByBidderId(String bidderId): Đọc lịch sử theo người dùng.
- getHighestBid(String auctionId): Lấy giao dịch giá cao nhất.

4. Những điều cần lưu ý khi code:

- Nếu việc insert lịch sử là một phần của quy trình place bid, cần cân nhắc dùng cùng transaction để không lệch dữ liệu.
- Không nên để DAO này tự sửa current price của auction; trách nhiệm đó thuộc AuctionDAO hoặc service điều phối.
- Với dữ liệu lớn, nên bổ sung giới hạn số bản ghi và bộ lọc thời gian.

5. Thứ tự ưu tiên:

Bước 5 — viết sau khi BidTransaction model và BidTransactionDAO đã xong.

ItemDAOImpl.java

1. Ý nghĩa & vai trò:

Triển khai SQL cho thực thể Item và các lớp con. Đây là class khó nhất của nhánh item vì nó phải xử lý ánh xạ kế thừa và bind đúng cột cho từng loại mặt hàng.

2. Kiến thức cần học trước khi code:

- Single-table inheritance và các trade-off khi một bảng chứa nhiều cột đặc thù.
- PreparedStatement, Types.NULL và ánh xạ giá trị null đúng kiểu dữ liệu.
- Factory/Mapper pattern để dựng đúng subclass từ category.
- Tổ chức code JDBC sao cho phần switch/instanceof không trở thành điểm nghẽn bảo trì.

3. Nội dung cần implement:

- mapRowToItem(ResultSet rs): Nhận category và dựng đúng object con của Item.
- findById(...), findAll(), findByCategory(...), findByOwnerId(...): Các truy vấn đọc item.

- `insert(Item item)`: Thêm mới item và bind đúng các cột đặc thù theo subclass.
- `update(Item item)`: Cập nhật metadata cần cho từng item.
- `delete(String id)`: Xóa item.
- `executeQueryList` helpers: Tái sử dụng logic query/read list mà không lặp code.

4. Những điều cần lưu ý khi code:

- Hiện tại code đang lệch schema ở cột `current_price` của items; phải sửa mô hình dữ liệu trước rồi mới viết DAO thật.
- Không nên set tất cả null bằng `Types.VARCHAR` cho mọi cột; int, boolean, decimal cần đúng kiểu để tránh driver xử lý mơ hồ.
- Nếu số danh mục tiếp tục tăng, nên tách mapper riêng hoặc dùng registry thay vì switch lớn.

5. Thứ tự ưu tiên:

Bước 5 — triển khai sau `ItemDAO`, toàn bộ model item con và quyết định cuối cùng về schema items.

UserDAOImpl.java

1. Ý nghĩa & vai trò:

Triển khai SQL cho người dùng đa hình. File này map role trong database thành Admin, Seller hoặc Bidder và thực thi các thao tác CRUD liên quan tới tài khoản.

2. Kiến thức cần học trước khi code:

- Polymorphic mapping theo role.
- `PreparedStatement`, `ResultSet` và cách xử lý cột nullable theo từng vai trò.
- Password hashing và xác minh tài khoản.
- Nguyên tắc không để DAO lẫn logic nghiệp vụ như phân quyền hoặc nạp tiền.

3. Nội dung cần implement:

- `mapRowToUser(ResultSet rs)`: Dựng đúng subclass User từ role.
- `findById(String id)`: Lấy người dùng theo ID.
- `findByUsername(String username)`: Lấy người dùng theo username.
- `insert(User user)`: Lưu tài khoản mới với các cột đặc thù phù hợp role.
- `update(User user)`: Cập nhật profile hợp lệ.
- `findAll()`: Đọc toàn bộ danh sách người dùng.

4. Những điều cần lưu ý khi code:

- Không lưu mật khẩu thô trong model rồi ghi thẳng xuống DB; phải hash ở service hoặc util trước khi gọi DAO.
- Role-specific field cần được cập nhật đầy đủ; không nên để Admin hoặc Seller bị bỏ sót khi update.
- Nên dùng Optional hoặc exception rõ ràng thay cho null để giảm lỗi NPE ở tầng service.

5. Thứ tự ưu tiên:

Bước 5 — triển khai sau `UserDAO`, `User` model và utility hash/validation.

Package model

Package giàu domain object, nhưng đang lẫn presentation console, String literal cho status/role và trùng trách nhiệm dữ liệu.

Entity.java

1. Ý nghĩa & vai trò:

Là lớp gốc cho mọi thực thể có định danh. Nó chuẩn hóa cách quản lý ID, thời điểm tạo và các hành vi chung như equals/hashCode/toString tối thiểu.

2. Kiến thức cần học trước khi code:

- Abstract class, inheritance và phạm vi truy cập protected/private.
- UUID, LocalDateTime và immutable design.
- equals/hashCode trong Java và tác động tới collection.
- Thiết kế base entity gọn, tránh để lớp cha ôm quá nhiều trách nhiệm.

3. Nội dung cần implement:

- Các field id, createdAt: Lưu định danh và thời gian tạo.
- Constructor cho object mới: Tự sinh UUID và createdAt khi tạo mới.
- Constructor cho object nạp từ DB: Nhận id, createdAt đã tồn tại.
- Getter cho field chung: Cho phép lớp khác đọc nhưng không sửa tùy tiện.
- equals()/hashCode(): So sánh thực thể theo id.

4. Những điều cần lưu ý khi code:

- Không nên gán createdAt = now cho object nạp từ DB, vì sẽ làm sai lịch sử dữ liệu.
- ID nên bất biến sau khi object được tạo.
- Base class chỉ nên giữ thuộc tính thật sự dùng chung cho mọi entity.

5. Thứ tự ưu tiên:

Bước 1 — làm đầu tiên vì hầu hết model khác đều kế thừa từ file này.

User.java

1. Ý nghĩa & vai trò:

Đại diện cho người dùng trừu tượng trong hệ thống. File này gom những trường chung của Admin, Seller và Bidder để các lớp con chỉ tập trung vào phần đặc thù.

2. Kiến thức cần học trước khi code:

- Abstract class, inheritance, encapsulation và polymorphism.
- Role-based design và enum thay cho String literal.
- Bảo mật mật khẩu, nguyên tắc hash + salt.
- Thiết kế model miền nghiệp vụ tách khỏi giao diện console.

3. Nội dung cần implement:

- Field chung: username, passwordHash, fullName, email, role, balance, status nếu cần.
- Constructor tạo mới và constructor nạp từ DB: Phục vụ hai bối cảnh khác nhau.
- Getter/Setter hợp lệ: Cập nhật dữ liệu có kiểm tra ràng buộc.
- deposit()/withdraw(): Các hành vi tác động số dư ở mức domain cơ bản.
- validateBaseInfo(): Kiểm tra username, email, password trước khi lưu.
- displayRolePermissions() hoặc mô tả vai trò: Nếu dùng cho demo OOP thì có thể giữ dạng abstract, nhưng không gắn UI cứng.

4. Những điều cần lưu ý khi code:

- Không expose password ra ngoài dưới dạng raw string.
- Các trường tiền nên dùng BigDecimal thay cho double.

- Role nên là enum UserRole để tránh sai chính tả như BUYER/BIDDER.
- Model không nên in log hoặc nội dung giao diện trực tiếp.

5. Thứ tự ưu tiên:

Bước 1 — làm sớm sau Entity vì các lớp người dùng và DAO đều phụ thuộc vào nó.

Admin.java

1. Ý nghĩa & vai trò:

Biểu diễn quản trị viên hệ thống. File này nên chỉ giữ dữ liệu và đặc quyền của Admin, còn thao tác ban user hay gỡ item nên được service xử lý.

2. Kiến thức cần học trước khi code:

- Kế thừa từ User.
- SRP: phân biệt model dữ liệu với action nghiệp vụ cấp quản trị.
- Phân quyền và kiểm soát dữ liệu nhạy cảm.
- Validation cho mã quản trị hoặc cấp truy cập.

3. Nội dung cần implement:

- Field accessLevel, adminCode: Lưu thông tin đặc thù của admin.
- Constructor tạo mới và nạp từ DB: Tạo object ở cả hai ngữ cảnh.
- Getter/Setter: Truy cập dữ liệu đặc thù.
- validateAdminInfo(): Kiểm tra access level và admin code hợp lệ.

4. Những điều cần lưu ý khi code:

- Không nên cài hành vi như banUser() trong model nếu hành vi đó phải cập nhật nhiều bảng.
- adminCode là dữ liệu nhạy cảm, không nên log công khai.
- Nếu có nhiều cấp admin, nên dùng enum hoặc constant rõ nghĩa thay vì số magic.

5. Thứ tự ưu tiên:

Bước 2 — làm sau User.java khi cấu trúc người dùng cơ sở đã ổn định.

Bidder.java

1. Ý nghĩa & vai trò:

Đại diện cho người tham gia đấu giá. File này giữ thông tin vận chuyển, liên hệ và các rule cơ bản liên quan đến người bid.

2. Kiến thức cần học trước khi code:

- Kế thừa từ User và quản lý profile người mua.
- Validation địa chỉ, số điện thoại và số dư ví.
- Nguyên tắc không giữ dữ liệu truy vấn tạm thời trong model nếu DB mới là nguồn thật.
- Collection trong Java và defensive copy.

3. Nội dung cần implement:

- Field shippingAddress, phoneNumber: Lưu thông tin phục vụ giao hàng.
- Constructor tạo mới và nạp từ DB: Phục vụ đăng ký mới và mapping từ database.
- Getter/Setter: Cập nhật thông tin liên lạc.
- validateBidderInfo(): Kiểm tra dữ liệu giao hàng trước khi thanh toán.

- Các helper liên quan đến ví: Như kiểm tra đủ số dư trước khi bid.

4. Những điều cần lưu ý khi code:

- Danh sách item đã bid không nên là nguồn dữ liệu chính nếu không được persist.
- Các thao tác thay đổi số dư nên qua service/transaction để tránh sai lệch concurrent.
- Thông tin giao hàng là dữ liệu cá nhân, cần hạn chế log và expose.

5. Thứ tự ưu tiên:

Bước 2 — làm sau User.java để chuẩn bị cho Auction và BidTransaction.

Seller.java

1. Ý nghĩa & vai trò:

Đại diện cho người bán trong hệ thống. File này lưu hồ sơ shop, điểm uy tín và các thông tin nền tảng cần cho việc đăng bán item.

2. Kiến thức cần học trước khi code:

- Kế thừa từ User và mô hình hóa seller profile.
- Validation tên shop, rating và thông tin liên hệ.
- Defensive copy cho collection.
- Phân biệt dữ liệu tổng hợp với dữ liệu truy vấn động.

3. Nội dung cần implement:

- Field storeName, rating: Giữ dữ liệu đặc thù của seller.
- Constructor tạo mới và nạp từ DB: Khởi tạo object cho cả hai trường hợp.
- Getter/Setter: Cập nhật thông tin shop.
- validateSellerInfo(): Kiểm tra tên shop và rating hợp lệ.

4. Những điều cần lưu ý khi code:

- Danh sách item đang bán không nên là nguồn thật nếu không đồng bộ với DB.
- rating là dữ liệu tính toán từ giao dịch, không nên để model tự sửa tùy tiện.
- Nếu có quy trình khóa shop hoặc duyệt seller, nên thêm status riêng thay vì nhồi vào rating.

5. Thứ tự ưu tiên:

Bước 2 — làm sau User.java và trước Item/Auction vì seller là người tạo auction.

Item.java

1. Ý nghĩa & vai trò:

Là lớp cha trừu tượng của mọi vật phẩm được đưa lên đấu giá. Nó gom phần metadata chung như tên, mô tả, giá khởi điểm, category và owner.

2. Kiến thức cần học trước khi code:

- Abstract class, inheritance và thiết kế hierarchy cho domain model.
- Validation dữ liệu đầu vào cho sản phẩm.
- Phân biệt rõ ranh giới giữa Item và Auction.
- Thiết kế model thuần, không gắn logic hiển thị console.

3. Nội dung cần implement:

- Field chung: name, description, startingPrice, category, ownerId và các trường thật sự cần dùng.

- Constructor tạo mới và nạp từ DB: Cho phép DAO và service dùng đúng ngữ cảnh.
- Getter/Setter: Truy cập có kiểm soát.
- validateBaseFields(): Kiểm tra tên, mô tả, giá khởi điểm, category.
- abstract summary()/typeName(): Nếu muốn đa hình, nên trả thông tin domain thay vì in console.

4. Những điều cần lưu ý khi code:

- currentPrice nhiều khả năng thuộc Auction chứ không thuộc Item; cần chốt mô hình để tránh lệch schema.
- Không nên chứa logic giao diện như showInfo() nếu đây là backend thuần.
- Category nên dùng enum ItemCategory để giảm lỗi string literal.

5. Thứ tự ưu tiên:

Bước 1 — làm sớm cùng User.java vì các item con và factory đều phụ thuộc vào lớp này.

Art.java

1. Ý nghĩa & vai trò:

Đại diện cho loại mặt hàng tác phẩm nghệ thuật trong miền nghiệp vụ. File này chứa các thuộc tính đặc thù (artist, creationYear, medium, dimensions, style, isOriginal, exhibitionHistory) để hệ thống có thể lưu, validate và hiển thị đúng thông tin cho từng danh mục đấu giá.

2. Kiến thức cần học trước khi code:

- Kế thừa trong Java, abstract class và cách thiết kế model thuần nghiệp vụ.
- Encapsulation, constructor overloading, getter/setter và quy tắc validation dữ liệu.
- Phân biệt dữ liệu của item với dữ liệu của auction để tránh trùng trách nhiệm.
- Ánh xạ object sang MySQL trong mô hình single-table hoặc joined-table inheritance.

3. Nội dung cần implement:

- Constructor khởi tạo mới: Nhận đủ dữ liệu đầu vào cần cho loại mặt hàng này.
- Constructor nạp từ database: Dùng khi DAO dựng object từ dữ liệu đã lưu.
- Getter/Setter cho thuộc tính đặc thù: Cho phép truy cập và cập nhật dữ liệu hợp lệ.
- validateSpecificFields(): Kiểm tra các ràng buộc riêng của danh mục trước khi lưu.
- toString()/summary(): Trả về chuỗi tóm tắt phục vụ log hoặc debug, không gắn UI console.
- isVintageArt(): Xác định nhanh tác phẩm cổ hoặc lâu năm nếu cần báo cáo.

4. Những điều cần lưu ý khi code:

- Không đặt logic in console hoặc logic giao diện vào model; model chỉ nên giữ dữ liệu và rule nghiệp vụ nhỏ.
- Nếu trường có thể vắng mặt, cần phân biệt rõ null hợp lệ với dữ liệu lỗi.
- Các trường tiền nên dùng BigDecimal thay cho double nếu tiến tới bản production.
- creationYear cần có khoảng hợp lệ; không nên chấp nhận năm vô lý hoặc lớn hơn năm hiện tại.
- dimensions và artist nên có quy tắc không rỗng nếu sản phẩm được phân loại là Art.
- Nếu có dữ liệu chứng thực tác phẩm, nên cân nhắc một object riêng thay vì dồn hết vào chuỗi tự do.

5. Thứ tự ưu tiên:

Bước 2 — triển khai sau các base model như Entity, User, Item để các lớp con bám đúng khung dữ liệu.

Collectible.java

1. Ý nghĩa & vai trò:

Đại diện cho loại mặt hàng đồ sưu tầm trong miền nghiệp vụ. File này chứa các thuộc tính đặc thù (yearOfOrigin, rarityLevel, conditionGrade, categorySpecific, hasCertificate, origin, specialFeatures) để hệ thống có thể lưu, validate và hiển thị đúng thông tin cho từng danh mục đấu giá.

2. Kiến thức cần học trước khi code:

- Kế thừa trong Java, abstract class và cách thiết kế model thuần nghiệp vụ.
- Encapsulation, constructor overloading, getter/setter và quy tắc validation dữ liệu.
- Phân biệt dữ liệu của item với dữ liệu của auction để tránh trùng trách nhiệm.
- Ánh xạ object sang MySQL trong mô hình single-table hoặc joined-table inheritance.

3. Nội dung cần implement:

- Constructor khởi tạo mới: Nhận đủ dữ liệu đầu vào cần cho loại mặt hàng này.
- Constructor nạp từ database: Dùng khi DAO dựng object từ dữ liệu đã lưu.
- Getter/Setter cho thuộc tính đặc thù: Cho phép truy cập và cập nhật dữ liệu hợp lệ.
- validateSpecificFields(): Kiểm tra các ràng buộc riêng của danh mục trước khi lưu.
- toString()/summary(): Trả về chuỗi tóm tắt phục vụ log hoặc debug, không gắn UI console.
- isCertifiedRareItem(): Tính nhanh xem đây có phải món hiếm đã kiểm định hay không.

4. Những điều cần lưu ý khi code:

- Không đặt logic in console hoặc logic giao diện vào model; model chỉ nên giữ dữ liệu và rule nghiệp vụ nhỏ.
- Nếu trường có thể vắng mặt, cần phân biệt rõ null hợp lệ với dữ liệu lỗi.
- Các trường tiền nên dùng BigDecimal thay cho double nếu tiến tới bản production.
- rarityLevel và conditionGrade nên có tập giá trị chuẩn hóa thay vì để string tự do.
- Nếu hasCertificate = true thì nên có thông tin chứng chỉ hoặc đơn vị xác thực đi kèm.
- Các mô tả hiếm/quý giá rất dễ bị nhập tự do thiếu nhất quán; nên chuẩn hóa enum hoặc reference data.

5. Thứ tự ưu tiên:

Bước 2 — triển khai sau các base model như Entity, User, Item để các lớp con bám đúng khung dữ liệu.

Electronics.java

1. Ý nghĩa & vai trò:

Đại diện cho loại mặt hàng đồ điện tử trong miền nghiệp vụ. File này chứa các thuộc tính đặc thù (brand, model, serialNumber, color, condition, warrantyMonths, isFullyFunctional, technicalSpec) để hệ thống có thể lưu, validate và hiển thị đúng thông tin cho từng danh mục đấu giá.

2. Kiến thức cần học trước khi code:

- Kế thừa trong Java, abstract class và cách thiết kế model thuần nghiệp vụ.
- Encapsulation, constructor overloading, getter/setter và quy tắc validation dữ liệu.
- Phân biệt dữ liệu của item với dữ liệu của auction để tránh trùng trách nhiệm.
- Ánh xạ object sang MySQL trong mô hình single-table hoặc joined-table inheritance.

3. Nội dung cần implement:

- Constructor khởi tạo mới: Nhận đủ dữ liệu đầu vào cần cho loại mặt hàng này.
- Constructor nạp từ database: Dùng khi DAO dựng object từ dữ liệu đã lưu.
- Getter/Setter cho thuộc tính đặc thù: Cho phép truy cập và cập nhật dữ liệu hợp lệ.
- validateSpecificFields(): Kiểm tra các ràng buộc riêng của danh mục trước khi lưu.

- `toString()/summary()`: Trả về chuỗi tóm tắt phục vụ log hoặc debug, không gắn UI console.
- `isUnderWarranty()`: Kiểm tra còn bảo hành hay không nếu có thêm thông tin ngày mua/đăng bán.

4. Những điều cần lưu ý khi code:

- Không đặt logic in console hoặc logic giao diện vào model; model chỉ nên giữ dữ liệu và rule nghiệp vụ nhỏ.
- Nếu trường có thể vắng mặt, cần phân biệt rõ null hợp lệ với dữ liệu lỗi.
- Các trường tiền nên dùng BigDecimal thay cho double nếu tiến tới bản production.
- `serialNumber` nên được kiểm tra trùng lặp nếu hệ thống dùng như định danh thực tế của thiết bị.
- `warrantyMonths` không được âm và `technicalSpec` cần có format dễ đọc.
- Trường `condition` nên đặt tên rõ như `conditionStatus` để tránh nhầm với từ khóa hoặc logic chung.

5. Thứ tự ưu tiên:

Bước 2 — triển khai sau các base model như Entity, User, Item để các lớp con bám đúng khung dữ liệu.

Fashion.java

1. Ý nghĩa & vai trò:

Đại diện cho loại mặt hàng thời trang trong miền nghiệp vụ. File này chứa các thuộc tính đặc thù (brand, size, material, color, gender, condition, isAuthentic) để hệ thống có thể lưu, validate và hiển thị đúng thông tin cho từng danh mục đấu giá.

2. Kiến thức cần học trước khi code:

- Kế thừa trong Java, abstract class và cách thiết kế model thuần nghiệp vụ.
- Encapsulation, constructor overloading, getter/setter và quy tắc validation dữ liệu.
- Phân biệt dữ liệu của item với dữ liệu của auction để tránh trùng trách nhiệm.
- Ánh xạ object sang MySQL trong mô hình single-table hoặc joined-table inheritance.

3. Nội dung cần implement:

- Constructor khởi tạo mới: Nhận đủ dữ liệu đầu vào cần cho loại mặt hàng này.
- Constructor nạp từ database: Dùng khi DAO dựng object từ dữ liệu đã lưu.
- Getter/Setter cho thuộc tính đặc thù: Cho phép truy cập và cập nhật dữ liệu hợp lệ.
- `validateSpecificFields()`: Kiểm tra các ràng buộc riêng của danh mục trước khi lưu.
- `toString()/summary()`: Trả về chuỗi tóm tắt phục vụ log hoặc debug, không gắn UI console.
- `requiresAuthenticationProof()`: Xác định item có cần kèm chứng từ xác thực khi đăng bán hay không.

4. Những điều cần lưu ý khi code:

- Không đặt logic in console hoặc logic giao diện vào model; model chỉ nên giữ dữ liệu và rule nghiệp vụ nhỏ.
- Nếu trường có thể vắng mặt, cần phân biệt rõ null hợp lệ với dữ liệu lỗi.
- Các trường tiền nên dùng BigDecimal thay cho double nếu tiến tới bản production.
- `size`, `gender` và `condition` nên dùng enum hoặc danh sách chuẩn để tránh dữ liệu nhập tự do.
- Nếu là hàng hiệu, thông tin xác thực cần rõ hơn một boolean đơn lẻ.
- Thông tin giới tính, size và tình trạng dễ thay đổi theo domain; nên chuẩn hóa để phục vụ filter.

5. Thứ tự ưu tiên:

Bước 2 — triển khai sau các base model như Entity, User, Item để các lớp con bám đúng khung dữ liệu.

Vehicle.java

1. Ý nghĩa & vai trò:

Đại diện cho loại mặt hàng phương tiện trong miền nghiệp vụ. File này chứa các thuộc tính đặc thù (brand, model, manufacturingYear, odo, engineType, color, licensePlate, hasLegalPapers, transmission) để hệ thống có thể lưu, validate và hiển thị đúng thông tin cho từng danh mục đấu giá.

2. Kiến thức cần học trước khi code:

- Kế thừa trong Java, abstract class và cách thiết kế model thuần nghiệp vụ.
- Encapsulation, constructor overloading, getter/setter và quy tắc validation dữ liệu.
- Phân biệt dữ liệu của item với dữ liệu của auction để tránh trùng trách nhiệm.
- Ánh xạ object sang MySQL trong mô hình single-table hoặc joined-table inheritance.

3. Nội dung cần implement:

- Constructor khởi tạo mới: Nhận đủ dữ liệu đầu vào cần cho loại mặt hàng này.
- Constructor nạp từ database: Dùng khi DAO dựng object từ dữ liệu đã lưu.
- Getter/Setter cho thuộc tính đặc thù: Cho phép truy cập và cập nhật dữ liệu hợp lệ.
- validateSpecificFields(): Kiểm tra các ràng buộc riêng của danh mục trước khi lưu.
- toString()/summary(): Trả về chuỗi tóm tắt phục vụ log hoặc debug, không gắn UI console.
- isRoadReady(): Đánh giá nhanh xe có đủ điều kiện pháp lý và vận hành để đấu giá.

4. Những điều cần lưu ý khi code:

- Không đặt logic in console hoặc logic giao diện vào model; model chỉ nên giữ dữ liệu và rule nghiệp vụ nhỏ.
- Nếu trường có thể vắng mặt, cần phân biệt rõ null hợp lệ với dữ liệu lỗi.
- Các trường tiền nên dùng BigDecimal thay cho double nếu tiến tới bản production.
- manufacturingYear và odo phải có ngưỡng hợp lệ; không nhận số âm hoặc năm vượt hiện tại.
- Nếu hasLegalPapers = true, nên có metadata về giấy tờ thay vì chỉ cờ boolean.
- licensePlate là dữ liệu nhạy cảm, có thể cần che bớt khi trả ra ngoài hệ thống.

5. Thứ tự ưu tiên:

Bước 2 — triển khai sau các base model như Entity, User, Item để các lớp con bám đúng khung dữ liệu.

Auction.java**1. Ý nghĩa & vai trò:**

Là thực thể lõi mô tả một phiên đấu giá. File này quản lý quan hệ với item, seller, bidder dẫn đầu, thời gian bắt đầu/kết thúc và trạng thái vòng đời phiên.

2. Kiến thức cần học trước khi code:

- Thiết kế aggregate root trong domain model.
- LocalDateTime, Duration và xử lý thời gian trong Java.
- Enum cho AuctionStatus và validation vòng đời trạng thái.
- Quy tắc đấu giá: giá mới phải cao hơn giá hiện tại, không bid khi hết hạn, không bid khi trạng thái không cho phép.

3. Nội dung cần implement:

- Field lõi: itemId, sellerId, currentHighestBidderId, currentPrice, startTime, endTime, status.
- Constructor tạo mới: Khởi tạo phiên với giá mở đầu và thời gian kết thúc.
- Constructor nạp từ DB: Dùng cho DAO mapping.
- isExpired(): Kiểm tra phiên đã quá hạn chưa.

- `canBid()`: Kiểm tra có thể nhận bid ở thời điểm hiện tại hay không.
- `applyBid(...)`: Cập nhật bidder dẫn đầu và giá mới ở mức domain.
- `close()/markPaid()/cancel()`: Các hành vi chuyển trạng thái có kiểm soát.

4. Những điều cần lưu ý khi code:

- Status hiện đang lệch mạnh với schema; phải dùng enum và chốt vòng đời thống nhất trước khi code tiếp.
- `currentPrice` phải dùng `BigDecimal` nếu yêu cầu độ chính xác tiền tệ cao.
- Không nên để entity tự truy cập DB hoặc tự xử lý thanh toán.
- Cần validate `endTime > startTime` và seller không được tự bid vào phiên của mình.

5. Thứ tự ưu tiên:

Bước 2 — làm sau `Item` và `Seller/Bidder` vì đây là thực thể trung tâm của nghiệp vụ đấu giá.

BidTransaction.java

1. Ý nghĩa & vai trò:

Biểu diễn một bản ghi lịch sử đặt giá. File này nên là object gần như immutable để phục vụ audit và truy vết giao dịch.

2. Kiến thức cần học trước khi code:

- Immutable object và audit trail.
- `LocalDateTime/Timestamp`, `UUID` và cách lưu lịch sử giao dịch.
- `BigDecimal` cho số tiền đặt giá.
- Thiết kế record-like model trong Java.

3. Nội dung cần implement:

- Field `auctionId`, `bidderId`, `bidAmount`, `bidTime`: Lưu dữ liệu của một lần bid.
- Constructor tạo mới: Sinh ID và thời gian ghi nhận.
- Constructor nạp từ DB: Dùng cho DAO mapping.
- Getter: Truy cập dữ liệu giao dịch.
- `isValidBid()`: Kiểm tra nhanh giao dịch có hợp lệ tối thiểu hay không.

4. Những điều cần lưu ý khi code:

- Không nên cho phép sửa `bidAmount` hoặc `bidderId` sau khi transaction đã tạo.
- Thời gian nên lấy từ server hoặc database để bảo đảm nhất quán.
- `BidTransaction` không thay thế trách nhiệm quản lý current price của Auction.

5. Thứ tự ưu tiên:

Bước 3 — làm sau `Auction.java` vì đây là dữ liệu phụ trợ nhưng rất quan trọng cho audit.

AutoBidding.java

1. Ý nghĩa & vai trò:

Mô tả cấu hình auto-bid của một bidder trên một phiên. File này dùng để lưu giới hạn tối đa, bước tăng và trạng thái hoạt động của cơ chế đấu giá tự động.

2. Kiến thức cần học trước khi code:

- Business rule cho auto-bid: max limit, increment, activation flag.
- Thiết kế entity cấu hình và cách liên kết nó với Auction/Bidder.
- `BigDecimal` cho giới hạn và bước tăng giá.

- Tư duy event-driven hoặc observer/scheduler để kích hoạt auto-bid đúng lúc.

3. Nội dung cần implement:

- Field auctionId, bidderId, maxLimit, incrementAmount, isActive: Lưu cấu hình auto-bid.
- Constructor tạo mới và nạp từ DB: Phục vụ service và DAO.
- calculateNextBid(BigDecimal currentAuctionPrice): Tính mức bid kế tiếp nếu chưa chạm trần.
- activate()/deactivate(): Bật hoặc tắt cấu hình auto-bid.
- validateConfig(): Kiểm tra maxLimit > current price và increment > 0.

4. Những điều cần lưu ý khi code:

- Model này vô nghĩa nếu không có DAO và service đi kèm; cần bổ sung AutoBiddingDAO/AutoBiddingService.
- Không nên để auto-bid tự ý đặt giá trực tiếp vào DB từ model.
- Cần tránh kích bản bidder tự auto-bid đối đầu với chính mình hoặc gây vòng lặp notify.

5. Thứ tự ưu tiên:

Bước 3 — chỉ làm sau khi core bidding đã ổn định; nếu dự án còn ít thời gian có thể ưu tiên sau cùng trong nhóm model.

Package factory

Phù hợp mục tiêu học design pattern, tuy nhiên đang khá verbose vì mỗi factory giữ nhiều state và tham số dài.

ItemFactory.java

1. Ý nghĩa & vai trò:

Đóng vai trò Factory Method để tạo Item từ dữ liệu đầu vào. File này giúp gom logic khởi tạo object theo danh mục vào một nơi thay vì rải rác trong service hoặc UI.

2. Kiến thức cần học trước khi code:

- Factory Method pattern và lợi ích của việc ẩn chi tiết khởi tạo object.
- Interface, polymorphism và cách trả về kiểu cha nhưng tạo ra kiểu con cụ thể.
- Validation dữ liệu đầu vào trước khi tạo object domain.
- Trade-off giữa factory tĩnh, factory có state và builder pattern.

3. Nội dung cần implement:

- Constructor hoặc request object: Nhận các trường đầu vào cần thiết để tạo item.
- createItem(): Tạo object Item từ bộ dữ liệu đã được kiểm tra.
- validateInput(): Kiểm tra dữ liệu đặc thù của Item: category, dữ liệu chung và dữ liệu đặc thù của từng subclass.

4. Những điều cần lưu ý khi code:

- Không nên để constructor của factory quá dài; nếu số tham số lớn nên gom vào DTO/request object.
- Factory chỉ tạo object, không nên tự đi lưu database hoặc xử lý giao dịch.
- Nếu factory được tái sử dụng nhiều nơi, cần tránh giữ mutable state gây lỗi khó debug.
- Nếu sau này có nhiều loại item, cần nhắc thêm generic hoặc registry để tránh if/switch ở tầng trên.

5. Thứ tự ưu tiên:

Bước 3 — làm sau khi các model đã ổn định, trước khi viết service hoặc DAO dùng đến logic tạo item.

ArtFactory.java

1. Ý nghĩa & vai trò:

Đóng vai trò Factory Method để tạo Art từ dữ liệu đầu vào. File này giúp gom logic khởi tạo object theo danh mục vào một nơi thay vì rải rác trong service hoặc UI.

2. Kiến thức cần học trước khi code:

- Factory Method pattern và lợi ích của việc ẩn chi tiết khởi tạo object.
- Interface, polymorphism và cách trả về kiểu cha nhưng tạo ra kiểu con cụ thể.
- Validation dữ liệu đầu vào trước khi tạo object domain.
- Trade-off giữa factory tĩnh, factory có state và builder pattern.

3. Nội dung cần implement:

- Constructor hoặc request object: Nhận các trường đầu vào cần thiết để tạo item.
- createItem(): Tạo object Art từ bộ dữ liệu đã được kiểm tra.
- validateInput(): Kiểm tra dữ liệu đặc thù của Art: artist, creation year, medium, dimensions, style và tính nguyên bản.

4. Những điều cần lưu ý khi code:

- Không nên để constructor của factory quá dài; nếu số tham số lớn nên gom vào DTO/request object.
- Factory chỉ tạo object, không nên tự đi lưu database hoặc xử lý giao dịch.
- Nếu factory được tái sử dụng nhiều nơi, cần tránh giữ mutable state gây lỗi khó debug.
- Với số lượng tham số lớn, nên đổi sang ArtCreationRequest để factory dễ đọc hơn.

5. Thứ tự ưu tiên:

Bước 3 — làm sau khi các model đã ổn định, trước khi viết service hoặc DAO dùng đến logic tạo item.

CollectibleFactory.java

1. Ý nghĩa & vai trò:

Đóng vai trò Factory Method để tạo Collectible từ dữ liệu đầu vào. File này giúp gom logic khởi tạo object theo danh mục vào một nơi thay vì rải rác trong service hoặc UI.

2. Kiến thức cần học trước khi code:

- Factory Method pattern và lợi ích của việc ẩn chi tiết khởi tạo object.
- Interface, polymorphism và cách trả về kiểu cha nhưng tạo ra kiểu con cụ thể.
- Validation dữ liệu đầu vào trước khi tạo object domain.
- Trade-off giữa factory tĩnh, factory có state và builder pattern.

3. Nội dung cần implement:

- Constructor hoặc request object: Nhận các trường đầu vào cần thiết để tạo item.
- createItem(): Tạo object Collectible từ bộ dữ liệu đã được kiểm tra.
- validateInput(): Kiểm tra dữ liệu đặc thù của Collectible: year of origin, rarity level, chứng chỉ và đặc điểm quý hiếm.

4. Những điều cần lưu ý khi code:

- Không nên để constructor của factory quá dài; nếu số tham số lớn nên gom vào DTO/request object.
- Factory chỉ tạo object, không nên tự đi lưu database hoặc xử lý giao dịch.
- Nếu factory được tái sử dụng nhiều nơi, cần tránh giữ mutable state gây lỗi khó debug.
- Nên chuẩn hóa rarityLevel và conditionGrade trước khi factory tạo object.

5. Thứ tự ưu tiên:

Bước 3 — làm sau khi các model đã ổn định, trước khi viết service hoặc DAO dùng đến logic tạo item.

ElectronicsFactory.java

1. Ý nghĩa & vai trò:

Đóng vai trò Factory Method để tạo Electronics từ dữ liệu đầu vào. File này giúp gom logic khởi tạo object theo danh mục vào một nơi thay vì rải rác trong service hoặc UI.

2. Kiến thức cần học trước khi code:

- Factory Method pattern và lợi ích của việc ẩn chi tiết khởi tạo object.
- Interface, polymorphism và cách trả về kiểu cha nhưng tạo ra kiểu con cụ thể.
- Validation dữ liệu đầu vào trước khi tạo object domain.
- Trade-off giữa factory tĩnh, factory có state và builder pattern.

3. Nội dung cần implement:

- Constructor hoặc request object: Nhận các trường đầu vào cần thiết để tạo item.
- createItem(): Tạo object Electronics từ bộ dữ liệu đã được kiểm tra.
- validateInput(): Kiểm tra dữ liệu đặc thù của Electronics: serial number, tình trạng, bảo hành và technical spec.

4. Những điều cần lưu ý khi code:

- Không nên để constructor của factory quá dài; nếu số tham số lớn nên gom vào DTO/request object.
- Factory chỉ tạo object, không nên tự đi lưu database hoặc xử lý giao dịch.
- Nếu factory được tái sử dụng nhiều nơi, cần tránh giữ mutable state gây lỗi khó debug.
- Các trường kỹ thuật dài như technicalSpec có thể nên tách cấu trúc thay vì nhét vào một chuỗi lớn.

5. Thứ tự ưu tiên:

Bước 3 — làm sau khi các model đã ổn định, trước khi viết service hoặc DAO dùng đến logic tạo item.

FashionFactory.java

1. Ý nghĩa & vai trò:

Đóng vai trò Factory Method để tạo Fashion từ dữ liệu đầu vào. File này giúp gom logic khởi tạo object theo danh mục vào một nơi thay vì rải rác trong service hoặc UI.

2. Kiến thức cần học trước khi code:

- Factory Method pattern và lợi ích của việc ẩn chi tiết khởi tạo object.
- Interface, polymorphism và cách trả về kiểu cha nhưng tạo ra kiểu con cụ thể.
- Validation dữ liệu đầu vào trước khi tạo object domain.
- Trade-off giữa factory tĩnh, factory có state và builder pattern.

3. Nội dung cần implement:

- Constructor hoặc request object: Nhận các trường đầu vào cần thiết để tạo item.
- createItem(): Tạo object Fashion từ bộ dữ liệu đã được kiểm tra.
- validateInput(): Kiểm tra dữ liệu đặc thù của Fashion: size, material, gender, condition và thông tin xác thực.

4. Những điều cần lưu ý khi code:

- Không nên để constructor của factory quá dài; nếu số tham số lớn nên gom vào DTO/request object.
- Factory chỉ tạo object, không nên tự đi lưu database hoặc xử lý giao dịch.
- Nếu factory được tái sử dụng nhiều nơi, cần tránh giữ mutable state gây lỗi khó debug.
- Cần chuẩn hóa size/gender nếu hệ thống có lọc theo các trường này.

5. Thứ tự ưu tiên:

Bước 3 — làm sau khi các model đã ổn định, trước khi viết service hoặc DAO dùng đến logic tạo item.

VehicleFactory.java

1. Ý nghĩa & vai trò:

Đóng vai trò Factory Method để tạo Vehicle từ dữ liệu đầu vào. File này giúp gom logic khởi tạo object theo danh mục vào một nơi thay vì rải rác trong service hoặc UI.

2. Kiến thức cần học trước khi code:

- Factory Method pattern và lợi ích của việc ẩn chi tiết khởi tạo object.
- Interface, polymorphism và cách trả về kiểu cha nhưng tạo ra kiểu con cụ thể.
- Validation dữ liệu đầu vào trước khi tạo object domain.
- Trade-off giữa factory tĩnh, factory có state và builder pattern.

3. Nội dung cần implement:

- Constructor hoặc request object: Nhận các trường đầu vào cần thiết để tạo item.
- createItem(): Tạo object Vehicle từ bộ dữ liệu đã được kiểm tra.
- validateInput(): Kiểm tra dữ liệu đặc thù của Vehicle: năm sản xuất, odo, engine type, legal papers và transmission.

4. Những điều cần lưu ý khi code:

- Không nên để constructor của factory quá dài; nếu số tham số lớn nên gom vào DTO/request object.
- Factory chỉ tạo object, không nên tự đi lưu database hoặc xử lý giao dịch.
- Nếu factory được tái sử dụng nhiều nơi, cần tránh giữ mutable state gây lỗi khó debug.
- Dữ liệu pháp lý và biến số cần kiểm tra kỹ trước khi tạo object chính thức.

5. Thứ tự ưu tiên:

Bước 3 — làm sau khi các model đã ổn định, trước khi viết service hoặc DAO dùng đến logic tạo item.

Package db

Hiện mới có một file, chưa đủ cho config, pooling và transaction management theo hướng production.

DatabaseManager.java

1. Ý nghĩa & vai trò:

Là cửa vào duy nhất để lấy connection và quản lý transaction. Trong backend Java thuần, đây là file nền móng quyết định toàn bộ DAO có ổn định hay không.

2. Kiến thức cần học trước khi code:

- JDBC nâng cao: DriverManager, DataSource, Connection lifecycle, auto-commit.
- Connection pooling và lý do dùng HikariCP thay cho mở connection trực tiếp từng lần.
- Singleton hoặc static holder pattern cho hạ tầng dùng chung.
- Đọc cấu hình từ file properties và quản lý secrets.

3. Nội dung cần implement:

- loadConfig(): Đọc database.properties hoặc AppConfig để lấy URL, user, password.
- initDataSource(): Khởi tạo HikariCP hoặc DataSource chung nếu dự án dùng connection pool.
- getConnection(): Cấp phát connection cho DAO.
- beginTransaction(Connection conn): Tắt auto-commit và bắt đầu transaction.
- commitTransaction(Connection conn): Commit và khôi phục trạng thái connection.

- `rollbackTransaction(Connection conn)`: Rollback an toàn khi có lỗi.

4. Những điều cần lưu ý khi code:

- Không hardcode URL, username, password trong source code; phải đọc từ properties hoặc biến môi trường.
- Nếu đã khai báo HikariCP trong pom.xml thì nên dùng thật, tránh phụ thuộc thừa.
- Các helper transaction phải luôn đưa connection về trạng thái an toàn trước khi trả lại pool.
- Không log full connection string nếu chứa thông tin nhạy cảm.

5. Thứ tự ưu tiên:

Bước 1 — đây là file hạ tầng quan trọng nhất cho toàn bộ tầng DAO.

Package service

Đây là package yếu nhất về cấu trúc vì mới có một class lớn ôm quá nhiều nghiệp vụ.

AuctionManager.java

1. Ý nghĩa & vai trò:

Đóng vai trò application service điều phối vòng đời phiên đấu giá. Nếu giữ file này, nó nên là một facade mong gọi tới các service/DAO chuyên trách thay vì ôm toàn bộ logic đặt giá, scheduler, observer và thanh toán.

2. Kiến thức cần học trước khi code:

- Service layer, orchestration và transaction boundary.
- Concurrency trong Java, scheduler và xử lý background task.
- Observer pattern, domain event và callback sau khi đặt giá thành công.
- Dependency injection thủ công trong Java thuần.

3. Nội dung cần implement:

- `createAuction(Auction auction)`: Tạo phiên mới và kích hoạt nếu hợp lệ.
- `getActiveAuctions()`: Trả về danh sách phiên đang mở.
- `getAuctionById(String auctionId)`: Lấy chi tiết phiên theo ID.
- `placeBid(User bidder, String auctionId, BigDecimal amount)`: Điều phối rule đặt giá và gọi DAO an toàn.
- `loadActiveAuctionsFromDB()`: Nạp cache hoặc danh sách active khi khởi động.
- `startAuctionEndMonitor()`: Khởi chạy scheduler kiểm tra phiên hết hạn.
- `closeAuction(Auction auction)`: Đóng phiên và chuyển sang bước thanh toán hoặc kết luận không có người mua.
- `processCheckoutTransaction(Auction auction)`: Xử lý settlement bằng transaction hoặc ủy quyền cho SettlementService.
- `addObserver/removeObserver/notifyObservers`: Quản lý danh sách observer nếu cần realtime/event.
- `shutdownManager()`: Đóng scheduler gọn gàng khi ứng dụng dừng.

4. Những điều cần lưu ý khi code:

- Phiên bản hiện tại đang quá tải trách nhiệm; về kiến trúc nên tách tối thiểu thành AuctionService, BiddingService, SettlementService và AuctionMonitorService.
- Không nên viết raw SQL trực tiếp trong service; service chỉ nên gọi DAO hoặc repository.
- Scheduler phải cẩn thận với race condition giữa thread monitor và người dùng đang bid phút cuối.
- Phần thanh toán cần transaction rõ ràng, log rõ ràng và cơ chế bù lỗi nếu thất bại.
- Không nên khởi tạo DAO implementation cứng trong constructor; hãy inject qua interface để dễ test.

5. Thứ tự ưu tiên:

Bước 6 — làm sau khi model, DAO contract và DAO implementation đã hoàn thiện.

Package util

Mới có bootstrap schema; còn thiếu các util quan trọng như hash password, validation, JDBC helper, time helper.

DatabaseSetup.java

1. Ý nghĩa & vai trò:

Dùng để khởi tạo database và chạy schema.sql trong môi trường local/dev. Đây là file tiện ích bootstrap, giúp dự án chạy nhanh mà không cần thao tác SQL tay.

2. Kiến thức cần học trước khi code:

- Đọc file từ classpath, InputStream, BufferedReader và xử lý encoding UTF-8.
- Thực thi DDL bằng JDBC.
- Idempotent schema setup và cách tách bootstrap cho môi trường phát triển.
- Quản lý cấu hình database qua properties.

3. Nội dung cần implement:

- `initDatabase()`: Tạo database nếu chưa có và chạy toàn bộ schema.sql.
- `loadConfig()`: Đọc database.properties.
- `readSchemaSQL()`: Đọc nội dung schema.sql từ resources.
- `splitStatements(String schema)`: Tách từng câu SQL cần thực thi.
- `buildAdminUrl(String dbUrl)`: Tạo URL kết nối tới cấp server để tạo database.
- `isIgnorableSchemaError(SQLException e)`: Bỏ qua một số lỗi idempotent khi chạy lại schema.

4. Những điều cần lưu ý khi code:

- Không nên hardcode tên database ở nhiều nơi; nên lấy từ config hoặc parse thống nhất từ db.url.
- Tách câu SQL chỉ bằng dấu ';' là giải pháp đơn giản nhưng sẽ lỗi khi có procedure/trigger phức tạp.
- File này phù hợp cho local bootstrap, không phải công cụ migration hoàn chỉnh cho production.
- Cần thống nhất với DatabaseManager về nguồn cấu hình để tránh mỗi nơi đọc một kiểu.

5. Thứ tự ưu tiên:

Bước 2 — làm ngay sau DatabaseManager để có thể dựng môi trường DB trước khi viết DAO.

3. Lộ Trình Học & Làm Theo Thứ Tự

Phần tổng hợp dưới đây ưu tiên cách triển khai ít tạo nợ kỹ thuật nhất cho một nhóm làm backend Java thuần. Mục tiêu là chốt hạ tầng và mô hình dữ liệu trước, sau đó mới viết DAO, rồi mới đến orchestration nghiệp vụ.

Thứ tự nên code các file

Thứ tự	File / nhóm file nên làm	Lý do
1	DatabaseManager.java, Entity.java, User.java, Item.java	Đây là nền móng hạ tầng và base model; sai ở đây sẽ kéo lệch toàn bộ DAO/service.
2	DatabaseSetup.java và chốt schema.sql + enum trạng thái/role/category	Cần đồng bộ dữ liệu trước khi viết DAO để tránh sửa hàng loạt về sau.
3	Admin.java, Bidder.java, Seller.java, Auction.java	Hoàn thiện các thực thể cốt lõi của người dùng và phiên đấu giá.
4	BidTransaction.java, AutoBidding.java và các item con: Art/Collectible/Electronics/Fashion/Vehicle	Bổ sung phần domain phụ trợ và danh mục sản phẩm sau khi base model ổn định.
5	ItemFactory.java và các factory cụ thể	Factory chỉ nên viết khi cấu trúc model đầu vào/đầu ra đã chốt.
6	UserDAO.java, ItemDAO.java, AuctionDAO.java, BidTransactionDAO.java và AutoBiddingDAO nếu bổ sung	Chốt contract persistence trước để implementation bám theo.
7	UserDAOImpl.java, ItemDAOImpl.java, AuctionDAOImpl.java, BidTransactionDAOImpl.java	Triển khai JDBC thật sau khi model và contract đã ổn định.
8	AuctionManager.java hoặc bộ service đã tách nhỏ	Chỉ nên orchestration khi các tầng dưới đã hoạt động và test được.
9	Utility còn thiếu: PasswordHasher, ValidationUtil, exception classes, tests	Tăng độ chắc chắn trước khi demo hoặc triển khai thêm chức năng nâng cao.

Lộ trình học kiến thức theo từng giai đoạn

Giai đoạn	Kiến thức cần học	Kết quả mong đợi
Giai đoạn 1	Java core, OOP, package, access modifier, abstract class, interface, enum, UUID, LocalDateTime, BigDecimal.	Hiểu được cấu trúc model, phân tầng package và viết được các base class sạch.
Giai đoạn 2	MySQL cơ bản, thiết kế schema, khóa ngoại, index, enum trong DB, chuẩn hóa dữ liệu.	Chốt được schema nhất quán với model và biết vì sao current price/status đang bị lệch.
Giai đoạn 3	JDBC: Connection, PreparedStatement, ResultSet, transaction, try-with-resources, SQLException handling.	Tự viết được DAO interface và DAO implementation ổn định.
Giai đoạn 4	DAO Pattern, Factory Method, Observer, SRP, dependency injection thủ công, mapper pattern.	Biết chia trách nhiệm đúng tầng, tránh đưa business logic hoặc UI vào model/DAO.

Giai đoạn 5	Nghiệp vụ đấu giá: vòng đời auction, bidding rules, settlement, auto-bidding, concurrency cơ bản.	Viết được service layer hợp lý và hiểu phần nào cần transaction hoặc locking.
Giai đoạn 6	Testing, logging, validation, password hashing, repo hygiene, refactor có kiểm soát.	Dự án đủ chắc để demo, debug và mở rộng thêm tính năng.

Kết luận ngắn

- Khung package hiện tại đủ để phát triển tiếp, nhưng bắt buộc phải chốt lại schema, enum và ranh giới trách nhiệm giữa Item với Auction trước khi thêm tính năng mới.
- Nếu chỉ sửa một chỗ duy nhất để giảm rủi ro, hãy tách AuctionManager và đồng bộ lại trạng thái auction với database.
- Nếu chỉ thêm một nhóm file mới, hãy ưu tiên enum + exception + AutoBiddingDAO/Service + PasswordHasher/Test.